

URL Building & Request Methods

url_for(), redirect(), GET, POST, PUT and DELETE

PFS-HYD-054 - Flask

```
return redirect(url_for('dashboard'))

@app.route('/login', methods=['GET', 'POST'])
def login():
    ...
```

Today students learn how Flask creates URLs and how browsers send different request methods.

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Today's learning map

The class moves from simple URL creation to real browser request methods.



1. Recall routing

URL, endpoint and view function from Day 3.

2. Build URLs

Use `url_for()` instead of hardcoding links.

3. Redirect users

Send users from one page to another correctly.

4. Understand HTTP methods

GET, POST, PUT and DELETE in web applications.

5. Write forms

Use GET and POST with a Flask route.

6. Practice APIs

Explain when PUT and DELETE are used.



Class outcome

By the end, students can build a mini login/search form and explain which HTTP method is used for each action.

Why URL building matters

Hardcoded links break silently when routes change. `url_for()` keeps links maintainable.



```
# Hardcoded link
<a href='/dashboard'>Dashboard</a>

# Better in Flask template
<a href='{ { url_for("dashboard") } }'>Dashboard</a>
```

Problem

Route changes from
/dashboard to /home.

Hardcoded

Every old link must be
edited manually.

`url_for()`

Only route
definition
changes.



Memory line

Do not write URLs everywhere. Give Flask the endpoint and let Flask create the URL.

url_for(): basic syntax

url_for() builds a URL using an endpoint name.



```
from flask import Flask, url_for

app = Flask(__name__)

@app.route('/')
def home():
    return 'Home Page'

@app.route('/about')
def about():
    return 'About Page'

with app.test_request_context():
    print(url_for('home'))
    print(url_for('about'))
```

Rule 1

Use the function name as the endpoint unless a custom endpoint is given.

Rule 2

url_for("home") returns / because home is mapped to /.

Rule 3

url_for("about") returns /about.

Rule 4

If endpoint spelling is wrong, Flask raises a BuildError.

Class activity

Ask students to predict output before running the code. Then change about() to company() and ask what breaks.

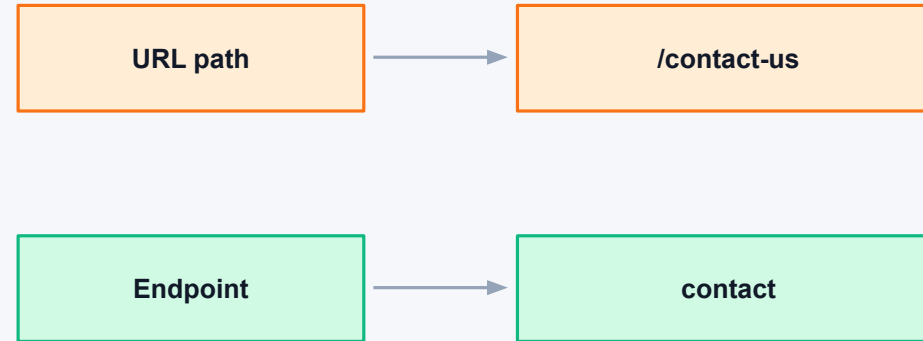
Endpoint versus URL path

Students must not confuse browser path with endpoint name.



```
@app.route('/contact-us')
def contact():
    return 'Contact Page'

url_for('contact')      # correct
url_for('contact-us')   # wrong
```



Exam-style trap

The endpoint is usually the function name, not the URL text written inside `@app.route()`.

Practical habit

Use descriptive function names because templates and redirects often depend on them.

url_for() with dynamic routes

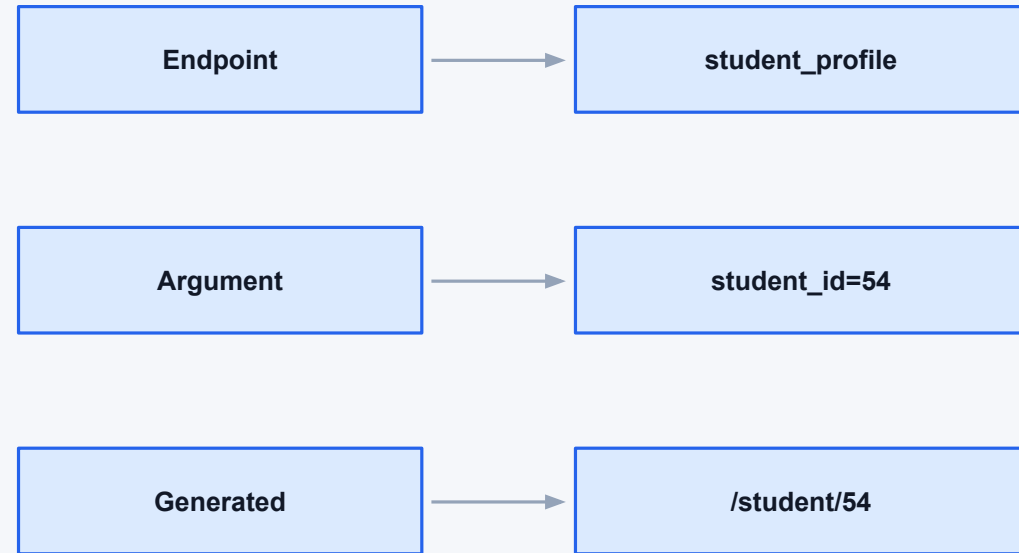
When a route has variables, url_for() must receive values for those variables.



```
@app.route('/student/<int:student_id>')
def student_profile(student_id):
    return f'Student ID: {student_id}'

# Build dynamic URL
url_for('student_profile', student_id=54)

# Output
'/student/54'
```



Common mistake

If the variable name is <student_id>, the argument must also be student_id. id=54 will not satisfy the route variable.

url_for() inside templates

Jinja templates use url_for() for routes and static files.



Page links

Use endpoint names for internal navigation.

Static files

Use endpoint static and filename argument.

```
<!-- templates/base.html -->
<a href="{{ url_for('home') }}">Home</a>
<a href="{{ url_for('about') }}">About</a>

<link rel="stylesheet"
      href="{{ url_for('static', filename='style.css') }}">


```

HTML

CSS

JS/Image

Trainer demo

Break a static path manually, then fix it with url_for("static", filename="...").

redirect(): sending the user to another route

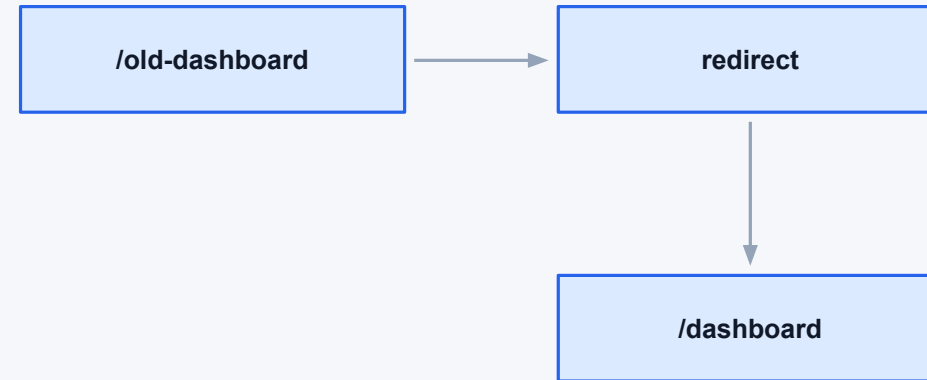
redirect() returns a response that tells the browser to visit a new URL.



```
from flask import redirect, url_for

@app.route('/old-dashboard')
def old_dashboard():
    return redirect(url_for('dashboard'))

@app.route('/dashboard')
def dashboard():
    return 'New Dashboard'
```



Use case

After login, go to dashboard.

Use case

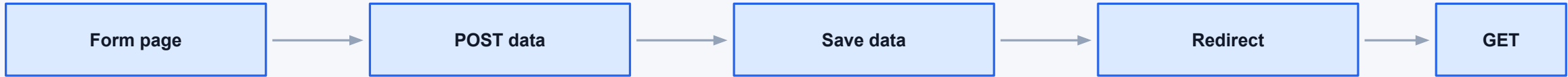
After form submit, avoid duplicate submit.

Memory line

url_for() builds the address. redirect() sends the browser to that address.

Redirect after form submit: PRG pattern

POST - Redirect - GET prevents duplicate form submission when the page refreshes.



```
@app.route('/save', methods=['POST'])
def save():
    # validate and store data
    return redirect(url_for('success'))

@app.route('/success')
def success():
    return 'Saved successfully'
```

Without redirect

Browser may warn: resubmit form?

With redirect

Refresh repeats GET page, not POST save.

Real project use

Registration, login, note creation, product creation and checkout flows usually redirect after POST.

HTTP methods overview

The method tells the server what the client wants to do.



GET	Purpose Read data	Example Search page, product list
POST	Purpose Create/submit data	Example Login, register, create note
PUT	Purpose Update/replace data	Example Update complete profile
DELETE	Purpose Delete data	Example Remove note or product

Important

Browsers use GET when you type a URL. HTML forms commonly use GET or POST. PUT and DELETE are usually practised through APIs or JavaScript clients.

GET request

GET is used to read or fetch information. Data can appear in the URL query string.



```
from flask import request

@app.route('/search')
def search():
    keyword = request.args.get('q')
    return f'Searching for {keyword}'

# URL
/search?q=python
```



Visible data

Query parameters are visible in the URL.

Best for

Search, filters, reading pages.

Do not use GET for passwords

Because data may appear in browser history, logs and bookmarks. Use POST for sensitive form submissions.

POST request

POST is used to submit data to the server. Form data is not placed directly in the URL.



```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')
        return 'Checking login details'
    return 'Show login form'
```

Form data

Access with request.form.

Use cases

Login, register, create item.

Safer than GET

Data is not shown in URL.

Still validate

Never trust input from browser.

Trainer warning

POST is not automatically secure. HTTPS, validation and password hashing are still required.

One route can handle GET and POST

A common Flask pattern: GET displays the form, POST processes the submitted form.

```
@app.route('/feedback', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        name = request.form.get('name')
        message = request.form.get('message')
        return redirect(url_for('thank_you'))

    return render_template('feedback.html')
```



Class check

Ask: What happens if methods=["GET", "POST"] is missing?

Live demo: simple registration form

Build a small form and print submitted values before introducing databases.



```
<!-- templates/register.html -->
<form method="POST">
  <input name="name" placeholder="Name">
  <input name="email" placeholder="Email">
  <button type="submit">Register</button>
</form>

# app.py
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form.get('name')
        email = request.form.get('email')
        return f'{name} - {email}'
    return render_template('register.html')
```

Step 1

Create templates/register.html.

Step 2

Create route with GET and POST methods.

Step 3

Read form values using request.form.get().

Step 4

Test empty fields and spelling mistakes in name attributes.

PUT request

PUT is generally used to update or replace an existing resource.



```
from flask import request, jsonify

@app.route('/api/student/<int:id>', methods=['PUT'])
def update_student(id):
    data = request.get_json()
    return jsonify({
        'message': 'student updated',
        'id': id,
        'data': data
    })
```

Usually API

PUT is commonly tested with Postman or JS fetch.

Update action

Use when an existing record changes.



Student note

HTML forms do not naturally support PUT. APIs and JavaScript clients are the normal practice environment.

DELETE request

DELETE is used to remove an existing resource.



```
from flask import jsonify

@app.route('/api/notes/<int:id>', methods=['DELETE'])
def delete_note(id):
    # delete note from database later
    return jsonify({
        'message': 'note deleted',
        'id': id
    })
```

Meaning

Remove existing data.

Example

Delete note, delete product,
remove cart item.

DELETE /api/notes/9

204 or JSON

Security warning

Before deleting in a real project, check login, ownership and confirmation.

Method behavior: safe and idempotent

This helps students reason about browser refresh and repeated API calls.



GET

Safe: should not change data.
Idempotent: repeated reads should be okay.

POST

Not idempotent: refreshing may create duplicate records.

PUT

Idempotent idea: same update repeated should produce same final state.

DELETE

Deleting the same item repeatedly should not create new data.

GET search

POST create

PUT update

DELETE remove

Exam-friendly line

GET reads, POST submits, PUT updates, DELETE removes.

Mini project: Student Request Methods App

A small classroom project to connect URL building, redirects and methods.



```
student_app/  
  app.py  
  templates/  
    base.html  
    home.html  
    register.html  
    search.html
```

Routes:

```
/           -> home  
/register    -> GET + POST  
/search      -> GET query  
/success     -> redirected page  
/api/student/1 -> PUT demo  
/api/student/1 -> DELETE demo
```

Task 1

Create navbar using `url_for()`.

Task 2

Create `/register` with GET and POST.

Task 3

After POST, redirect to `/success`.

Task 4

Create `/search?q=keyword` and read `request.args`.

Task 5

Explain PUT and DELETE using API route examples.

Common mistakes to catch quickly

These are likely to happen during Day 4 practice.



1. Using URL path inside url_for()

Wrong: `url_for("/about")`. Correct: `url_for("about")`.

2. Forgetting methods=["POST"]

The browser submits, but Flask responds with method not allowed.

3. Reading POST data from request.args

POST form values usually come from `request.form`.

4. Form input missing name attribute

Without name, browser does not submit that field key.

5. Redirecting to function call

Wrong: `redirect(dashboard())`. Better: `redirect(url_for("dashboard"))`.

6. Mismatched route variable

`<int:id>` requires `url_for("view", id=10)`.

7. Using GET for passwords

Passwords should be submitted with POST over HTTPS.

8. Not testing redirect flow

Always check final URL and browser behavior after submit.

Debugging habit

Read the error line first: `BuildError`, `Method Not Allowed`, `KeyError` and `None` values usually identify the issue fast.

Classroom practice plan

Use short rounds so students practise every concept before the exit test.



Round 1 - url_for()

Create home, about and contact links.

Round 2 - Dynamic URL

Build /student/<id> using url_for().

Round 3 - Redirect

Redirect /old-home to /.

Round 4 - GET

Create /search?q=flask.

Round 5 - POST

Create a feedback form.

Round 6 - Explain

Say when GET, POST, PUT and DELETE are used.

Exit assessment and homework

End the class with a short practical check and clear next-day preparation.



In-class assessment

1. Build /profile link using url_for().
2. Redirect /old-profile to /profile.
3. Create GET search route.
4. Create POST feedback route.
5. Explain GET vs POST.

Acceptance criteria

- Application runs without errors.
- No hardcoded internal links.
- POST form reads submitted values.
- Redirect changes browser URL correctly.

Homework

Create a Student Enquiry form using GET for search and POST for submission. Use url_for() in all navigation links.

GitHub evidence

Commit message: Completed Flask Day 4 URL Building and Methods.

Tomorrow: Request object, user input and basic validation.

Day 4 quick recap

Students should answer these before leaving.



1. Why should we use `url_for()` instead of hardcoded URLs?
2. What is the difference between a URL path and endpoint?
3. What does `redirect()` do?
4. When should GET be used?
5. When should POST be used?
6. What are PUT and DELETE mainly used for?
7. Why do we redirect after form submission?

```
return redirect(url_for('home'))  
  
url_for('static', filename='style.css')
```

Final memory line

`url_for` builds URLs. `redirect` sends browsers.
Methods describe actions.

Next class

Request object and user input handling.